CSC 251 Lecture 20: OOP and Inheritance

Python OOP Highlights

- In Python, a class can only contain one initializer (constructor)
- Always use self as the first (or only) parameter in all class methods. Forgetting `self` in front of attribute names creates a local variable
- Attributes are designated as **non-public** using __(two underscores)
- A `__str__` method can be used to output the object's state, overriding the unhelpful default message, invoked by printing the object:

```
print(object)
```

- The `__dict__` attribute creates a dictionary containing all attributes of the object, used to inspect objects with numerous attributes:

```
print(object.__dict__)
```

for our Coin class:

```
{'_Coin__sideup': 'Heads'}
```

Or:

```
print(Class.__dict__)
```

returns:

```
{'__module__': 'coins', '__doc__': "\n  A class representing a coin.\n  Attrib-
utes:\n    sideup (str): The value of the side facing up, 'Heads' or 'Tails'\n  ",
'__init__': <function Coin.__init__ at 0x103547060>, 'toss': <function Coin.toss at
0x103547100>, 'sideup': <property object at 0x103533ba0>, '__dict__': <attribute
'__dict__' of 'Coin' objects>, '__weakref__': <attribute '__weakref__' of 'Coin'
objects>}
```

- Methods can call other methods

```
def __str__(self):
    return f'The final sales price is ${self.computeTotalDiscount():,.2f}'
```

- Objects may be passed as arguments to functions that are not a part of the class.

Example 1: For a `Product` object called `gobstopper`:

```
def output_sale_price(gobstopper):
    gobstopper.discount = 10.00
    print("On sale now: ", gobstopper.compute_new_price())
```

Back in main, what does this imply?

```
output_sale_price(gobstopper)
```

Example 2: Another example with decorators and type hints:

```
class Student:
    def __init__(self, name: str = '', id: str = '0000000'):
        self.__name: str  = name
        self.__id: str = id

    @property
    def name(self) -> str:
        return self.__name

    @name.setter
    def name(self, sname) -> None:
        self.__name = sname.upper()

    @property
    def id(self) -> str:
        return self.__id

    @id.setter
    def id(self, studID)-> None:
        if len(studID) <= 7:
            self.__id = studID
        else:
            self.__id = studID[-7:]

    @name.deleter
    def name(self) -> None:
        print(f'{self.__name} is deleted.')
        del self.__name



cs_student1 = Student('Joe Schmoe')
cs_student1.id = '3920244'
print('ID: ', cs_student1.id)
print('Name: ', cs_student1.name)
cs_student1.id = '0001234567'
print('ID: ', cs_student1.id)
del cs_student1.name
```

Once the name attribute is deleted:

```
print(cs_student1.name)
```

```
AttributeError: 'Student' object has no attribute '_Student__name'.
```

Other Dunder Methods

For a user-friendly list:
https://www.geeksforgeeks.org/dunder-magic-methods-python/

For the complete documentation:
https://docs.python.org/3/reference/datamodel.html

To get a list of magic functions for a specific class, for example, for the `int` class:

```
dir(int)
```

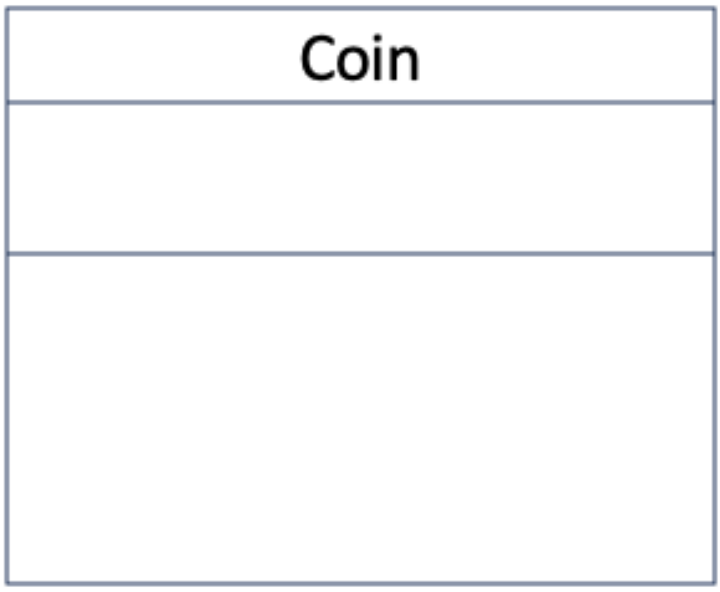
Class Design

What do you recall about class design?

Classes usually describe:

-
-
-
-

Example 3: Let's create a simplified UML diagram for our `Coin` class.



Note: NO appearance of "self" parameter

Inheritance

Inheritance allows one class to exist as a specialized version of another class, inheriting all the methods and properties from the class, demonstrating an "is a" relationship:

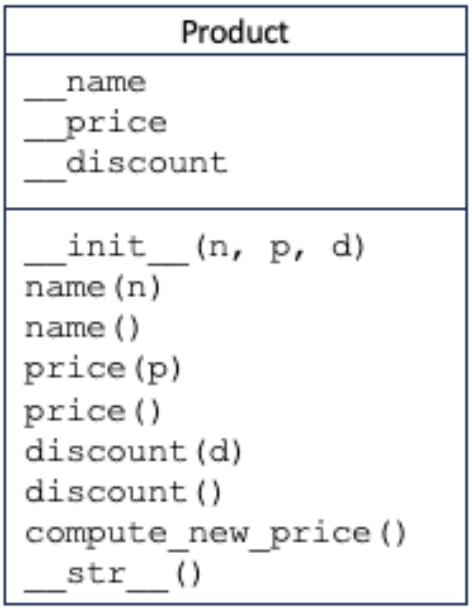
A poodle is a dog
A rose is a flower
A circle is a shape
A car is a vehicle
A soccer player is an athlete

Define multiple classes:

superclass –

subclass -

Example 4a: Our previously-defined class `Product` could be defined as a superclass:

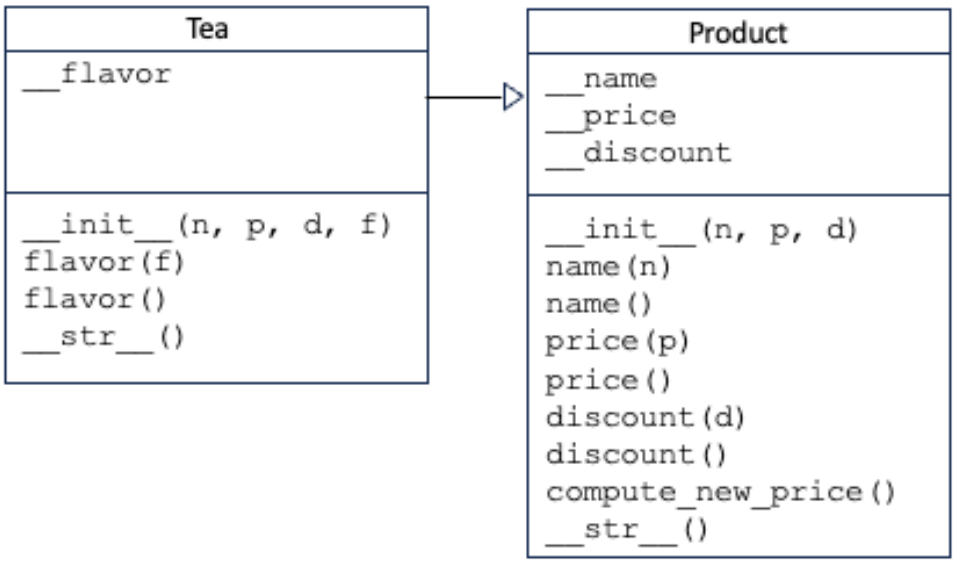


The class contains a constructor, a string method, a setter and getter for each attribute, and a method to compute the discounted price. How do we distinguish between the setter and getter?

Note: A deleter, if present, would be distinguished by:

`{delete} property_name()` in the UML.

Example 4b: Let's create a `Tea` subclass with an additional "flavor" attribute with a new getter and setter:



Example 4c: The `Product` super class from before:

```
class Product:

    def __init__(self, name = '', price = 0.0, discount = 0):
        self.__name = name
        self.__price = price
        self.__discount = discount

    @property
    def name(self):
        return self.__name

    @name.setter
    def name(self, n):
        self.__name = n

    @property
    def price(self):
        return self.__price

    @price.setter
    def price(self, p):
        self.__price = p

    @property
    def discount(self):
        return self.__discount

    @discount.setter
    def discount(self, d):
        self.__discount = d

    def compute_new_price(self):
        return f'$ {self.__price * (1 - self.__discount / 100):,.2f}'
```

```
    def __str__(self):
        return f'Product name: {self.__name}, Price: $ {self.__price:,.2f},
            Discount: {self.__discount}%'
```

Example 4d: The `Tea` subclass:

```
class Tea(Product):
    def __init__(self, name = '', price = 0.0, discount = 0, flavor = ''):
        super().__init__(name, price, discount)

        self.__flavor = flavor

    def __str__(self):
        return super().__str__() + f", flavor: {self.__flavor}"
      . . .
```

What indicates the relationship the `Tea` class and the `Product` superclass?

The constructor and `__str__` method from the `Product` class can be called using the `super()` expression. Another option in the constructor is to call the `Product` class directly, including the `self` parameter:

```
     Product.__init__(self, name, price, discount)
```

The `Tea` class also includes a setter and getter for the flavor attribute, not shown above.

Example 4e: In Main, the classes are imported from a file containing both class definitions:

```
from tea import Tea, Product

. . .
   cup = Tea("Bigelow 40 count tea bags", 4.99, 10, "Earl Grey")
   print(cup)
```

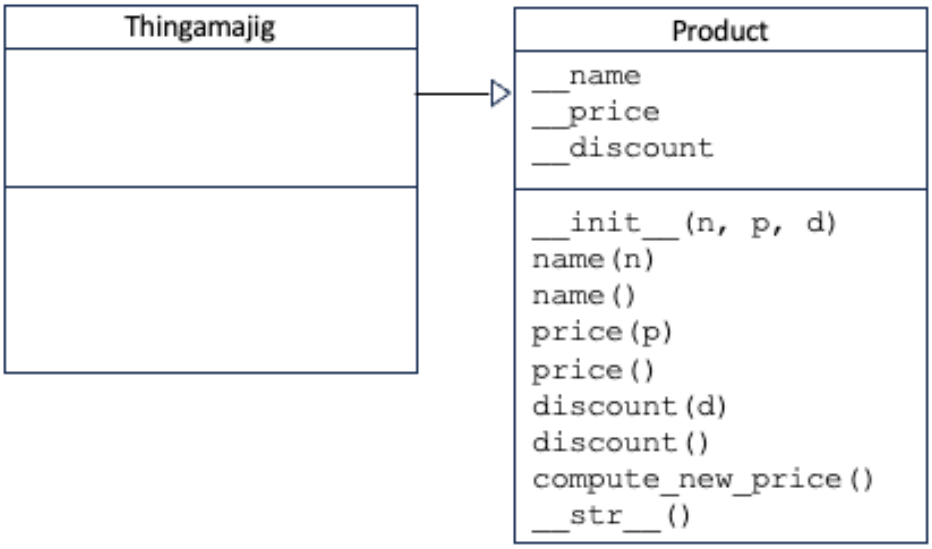
Outputs:

```
Product name: Bigelow 40 count tea bags, Price: $ 4.99, Discount: 10%, flavor: Earl Grey
```

Example 4f: We can include code dependent upon the object's type:

```
if isinstance(object, Tea):
     sugar = input( "Would you like sugar in your tea? (y or n)")
```

Example 5: If the subclass uses the `Product` class with no additional attributes or methods, the definition is still present with a `pass` keyword as a placeholder.




```
class Thingamajig(Product):
    pass
```